

Data Analysis using Python

05 – Merging and Grouping

Contents

Ranking

Concatenation

Merging

Joining

Grouping and aggregation

Ranking Data

Assigns a rank to each value in a dataset, based on its size relative to other values.
Handles ties by assigning the same rank or using specific tie-breaking methods.
Useful for identifying relative positions of data points.

Key Parameters of **rank()**:

method: Specifies the method to handle ties:

average (default): Assigns the average rank of tied groups.

min: Assigns the lowest rank for ties.

max: Assigns the highest rank for ties.

dense: Like min, but ranks increase by 1 for each group.

first: Ranks in the order of appearance.

ascending: Controls sorting order (default: True for ascending ranks).

na_option: How to handle NaN values (keep, top, or bottom).

Ranking Data

```
import pandas as pd

# Sample Premier League Table
data = {
    'Team': ['Manchester City', 'Liverpool', 'Arsenal', 'Manchester United', 'Tottenham'],
    'Points': [69, 85, 84, 70, 74],
    'Goal Difference': [63, 66, 45, 20, 29]
}
df = pd.DataFrame(data)

# Rank teams by points (descending)
df['Rank'] = df['Points'].rank(method='dense', ascending=False)

# Display the ranked table
print(df)
```

Concatenation

Used to combine two or more DataFrames or Series along a particular axis (rows or columns).
Useful for combining data from multiple sources or creating larger datasets.

Key Parameters of **pd.concat**:

objs: A list of DataFrames or Series to concatenate.

axis: The axis along which to concatenate:

axis=0 (default): Combines along rows (stacking vertically).

axis=1: Combines along columns (side by side).

keys: Adds hierarchical keys to differentiate between the concatenated objects.

ignore_index: Resets the index in the concatenated DataFrame.

Concatenation

```
import pandas as pd

# Sample DataFrames
data_1 = {
    'Team': ['Manchester City', 'Liverpool', 'Arsenal'],
    'Points': [89, 85, 84]
}
data_2 = {
    'Team': ['Manchester United', 'Tottenham'],
    'Points': [74, 70]
}

df1 = pd.DataFrame(data_1)
df2 = pd.DataFrame(data_2)

# Concatenate along rows
df_combined = pd.concat([df1, df2], axis=0, ignore_index=True)

print(df_combined)
```

	Team	Points
0	Manchester City	89
1	Liverpool	85
2	Arsenal	84
3	Manchester United	74
4	Tottenham	70

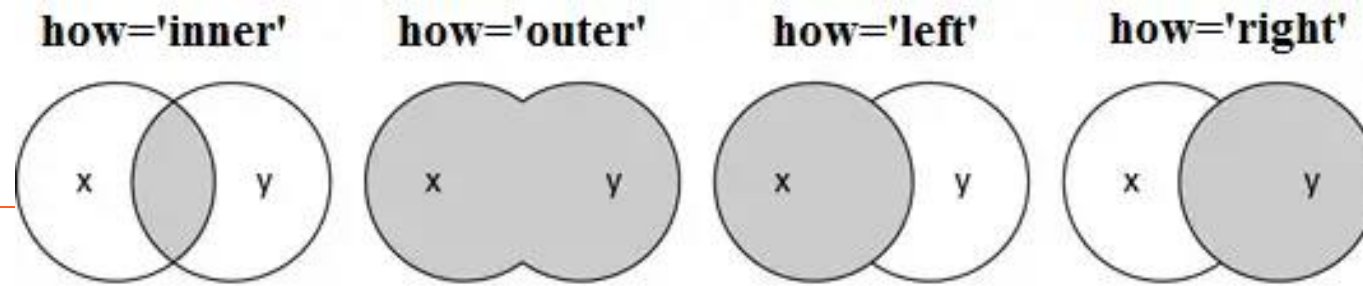
Concatenation

Concatenate along columns

df_combined = pd.concat([df1, df2], axis=1, ignore_index=True)

	0	1	2	3
0	Manchester City	89	Manchester United	74.0
1	Liverpool	85	Tottenham	70.0
2	Arsenal	84	NaN	NaN

Merging



Merging combines data from two or more DataFrames based on a key column or index.

Similar to SQL joins, Pandas supports left, right, outer, and inner merges.

Types of Merges

Left Merge

Keeps all rows from the left DataFrame and matches data from the right DataFrame. Unmatched rows in the right DataFrame result in NaN.

Right Merge

Keeps all rows from the right DataFrame and matches data from the left DataFrame. Unmatched rows in the left DataFrame result in NaN.

Outer Merge

Includes all rows from both DataFrames, with NaN for unmatched keys.

Inner Merge

Includes only rows with matching keys in both DataFrames.

Merging

```
import pandas as pd
```

```
# DataFrames
```

```
data_1 = {'Team': ['Manchester City', 'Liverpool', 'Arsenal'], 'Points': [89, 85, 84]}
```

```
data_2 = {'Team': ['Liverpool', 'Arsenal', 'Chelsea'], 'Goal Difference': [66, 45, 27]}
```

```
df1 = pd.DataFrame(data_1)
```

```
df2 = pd.DataFrame(data_2)
```

	Team	Points
0	Manchester City	89
1	Liverpool	85
2	Arsenal	84

df1

	Team	Goal Difference
0	Liverpool	66
1	Arsenal	45
2	Chelsea	27

df2

```
left_merge = pd.merge(df1, df2, on='Team', how='left')
```

```
print(left_merge)
```

	Team	Points	Goal Difference
0	Manchester City	89	NaN
1	Liverpool	85	66.0
2	Arsenal	84	45.0

```
right_merge = pd.merge(df1, df2, on='Team', how='right')
```

```
print(right_merge)
```

	Team	Points	Goal Difference
0	Liverpool	85.0	66
1	Arsenal	84.0	45
2	Chelsea	NaN	27

```
Outer_merge = pd.merge(df1, df2, on='Team', how='outer')
```

```
print(outer_merge)
```

	Team	Points	Goal Difference
0	Arsenal	84.0	45.0
1	Chelsea	NaN	27.0
2	Liverpool	85.0	66.0
3	Manchester City	89.0	NaN

```
Inner_merge = pd.merge(df1, df2, on='Team', how='inner')
```

```
print(inner_merge)
```

	Team	Points	Goal Difference
0	Liverpool	85	66
1	Arsenal	84	45

Joining

Joining combines two DataFrames based on their index rather than a specific column.
Useful when working with DataFrames that have overlapping or related indices.

Types of Joins in Pandas

Left Join

Includes all rows from the left DataFrame and matches data from the right DataFrame. Missing data results in NaN.

Right Join

Includes all rows from the right DataFrame and matches data from the left DataFrame. Missing data results in NaN.

Outer Join

Includes all rows from both DataFrames, filling in NaN for unmatched indices.

Inner Join

Includes only rows with matching indices in both DataFrames.

Joining

```
import pandas as pd
```

```
# DataFrames
```

```
df1 = pd.DataFrame({'Points': [89, 85, 84]}, index=['Manchester City', 'Liverpool', 'Arsenal'])
```

```
df2 = pd.DataFrame({'Goal Difference': [66, 45, 27]}, index=['Liverpool', 'Arsenal', 'Chelsea'])
```

```
print(df1)
```

	Points
Manchester City	89
Liverpool	85
Arsenal	84

```
print(df2)
```

	Goal Difference
Liverpool	66
Arsenal	45
Chelsea	27

```
left_join = df1.join(df2, how='left')
```

```
print(left_join)
```

	Points	Goal Difference
Manchester City	89	NaN
Liverpool	85	66.0
Arsenal	84	45.0

```
right_join = df1.join(df2, how='right')
```

```
print(right_join)
```

	Points	Goal Difference
Liverpool	85.0	66
Arsenal	84.0	45
Chelsea	NaN	27

```
outer_join = df1.join(df2, how='outer')
```

```
print(outer_join)
```

	Points	Goal Difference
Arsenal	84.0	45.0
Chelsea	NaN	27.0
Liverpool	85.0	66.0
Manchester City	89.0	NaN

```
inner_join = df1.join(df2, how='inner')
```

```
print(inner_join)
```

	Points	Goal Difference
Liverpool	85	66
Arsenal	84	45

Grouping and Aggregating

Grouping

Splitting data into groups based on some criteria.

Aggregating

Applying a function to each group (e.g., sum, mean, count) to summarize data.

Key Steps in Grouping and Aggregating:

1. Split the Data: Use **groupby()** to group data based on a column or multiple columns.
2. Apply Aggregation Functions: Use built-in functions like **sum()**, **mean()**, or custom functions.
3. Combine Results: Generate a summarized DataFrame.

Grouping and Aggregating

Example 1: Basic Grouping

```
import pandas as pd
```

Sample DataFrame

```
data = {  
    'Team': ['A', 'A', 'B', 'B', 'C', 'C'],  
    'Points': [10, 15, 20, 25, 30, 35],  
    'Region': ['East', 'East', 'West', 'West', 'East', 'West']  
}
```

```
df = pd.DataFrame(data)
```

Group by Team and calculate total points

```
grouped = df.groupby('Team')['Points'].sum()
```

```
print(grouped)
```



Team	
A	25
B	45
C	65

Grouping and Aggregating

Example 2: Grouping with Multiple Columns

Group by Region and Team, then calculate mean points

```
grouped_multi = df.groupby(['Region', 'Team'])['Points'].mean()
```

```
print(grouped_multi)
```

Region	Team	
East	A	12.5
	C	30.0
West	B	22.5
	C	35.0

Example 3: Aggregate with multiple functions

```
agg = df.groupby('Region')['Points'].agg(['sum', 'mean', 'max'])
```

```
print(agg)
```

Region	sum	mean	max
East	55	18.333333	30
West	80	26.666667	35

Grouping and Aggregating

Grouping by time series involves splitting data into time-based intervals (e.g., daily, monthly, yearly) for aggregation and analysis.

Commonly used for analyzing trends, seasonality, or performance over time.

```
import pandas as pd
import numpy as np

# Create sample time series data over 3 months
data = {
    'Date': pd.date_range(start='2024-01-01', end='2024-03-31', freq='D'),
    'Sales': np.random.randint(150, 350, size=91) # Random sales values between 150 and 350
}
df = pd.DataFrame(data)
df.set_index('Date', inplace=True)

# Group by month and calculate total sales
monthly_sales = df.resample('ME').sum()

print(monthly_sales)
```

Date	Sales
2024-01-31	7624
2024-02-29	7479
2024-03-31	7442

